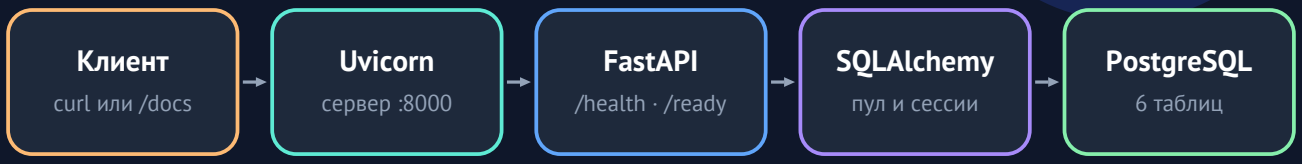
УЧЕБНЫЙ ПРОЕКТ № 2 · ВЫМЫШЛЕННАЯ КОМПАНИЯ INDUSTRIALFLOW

AI Data Analyst

Этапы 01–05 · API, база и аналитика

Наглядное руководство: окружение uv, API на FastAPI, PostgreSQL в Docker, миграции Alembic, SQL, воспроизводимые данные, аналитика, аномалии, тесты и CI.



Числа в отчётах считает код, а данные защищает сама база.

5 / 10	223 / 223	9 / 9	6	52
этапов курса завершены	автотестов прошли	локальных проверок PASS	таблиц PostgreSQL	пакетов в uv.lock

Что внутри 01: 2–11 · 02: 12–20 · 03: 21–23 · 04: 24–26 · 05: 27–29

1 Проект за минуту
2 Путь запроса
3 Окружение и uv
4 Создание проекта
5 Импорт и sys.path
6 HTTP и JSON
7 Разбор кода API
8 Настройки
9 Тесты API
10 Ruff и mypy
11 Git и GitHub
12 PostgreSQL в Docker
13 Таблицы и связи
14 Правила целостности
15 Деньги и время
16 SQLAlchemy и сессии
17 Миграции Alembic
18 /health и /ready
19 SQL вручную
20 Тесты с базой
21 Воспроизводимые данные
22 Сценарии A01–A05
23 Загрузка и качество
24 Точные показатели
25 Месяцы и сравнения
26 Вклады групп
27 Статистические кандидаты
28 Isolation Forest
29 Отдельная оценка сценариев
30 CI на GitHub Actions
31 Шпаргалка команд
32 Типичные ошибки
33 Проверь себя
34 Итоги проверки

Проверка 2026-10-21 · коммит 1aa1d24 + worktree · uv 0.12.23 · Python 3.12.15 · PostgreSQL 17 · macOS, Apple Silicon
Этапы: docs/STAGE_01.md–STAGE_05.md; отчёт: docs/VERIFICATION.md

1 Проект за минуту

Что уже работает, чего ещё нет и где проект в плане курса.

AI Data Analyst — ИИ-аналитик вымышленной компании **IndustrialFlow**, которая продаёт насосы, компрессоры, арматуру и датчики. Руководитель задаёт вопрос обычным языком, а агент вызывает аналитические инструменты, точно считает показатели и пишет отчёт. Готовы этапы 01–05: каркас API с проверками и CI, база PostgreSQL, данные, проверки качества, точная аналитика и поиск аномалий.

API

FastAPI: GET /health — процесс жив; GET /ready — база доступна и схема актуальна. Документация /docs строится сама.

База данных

PostgreSQL 17 в Docker Compose: 6 таблиц, 28 правил целостности, деньги в NUMERIC, время в timestamptz.

Миграции

Схема создаётся и откатывается только Alembic; alembic check ловит модель без миграции.

Тесты

223 тестов pytest: 136 unit и 87 на настоящей PostgreSQL. Все проходят.

Качество

Ruff проверяет стиль и форматирует код и примеры в Markdown; mypy в строгом режиме проверяет типы.

CI

GitHub Actions повторяет всё на чистой Linux-машине с отдельной PostgreSQL при каждом push и pull request.

Где мы в плане курса

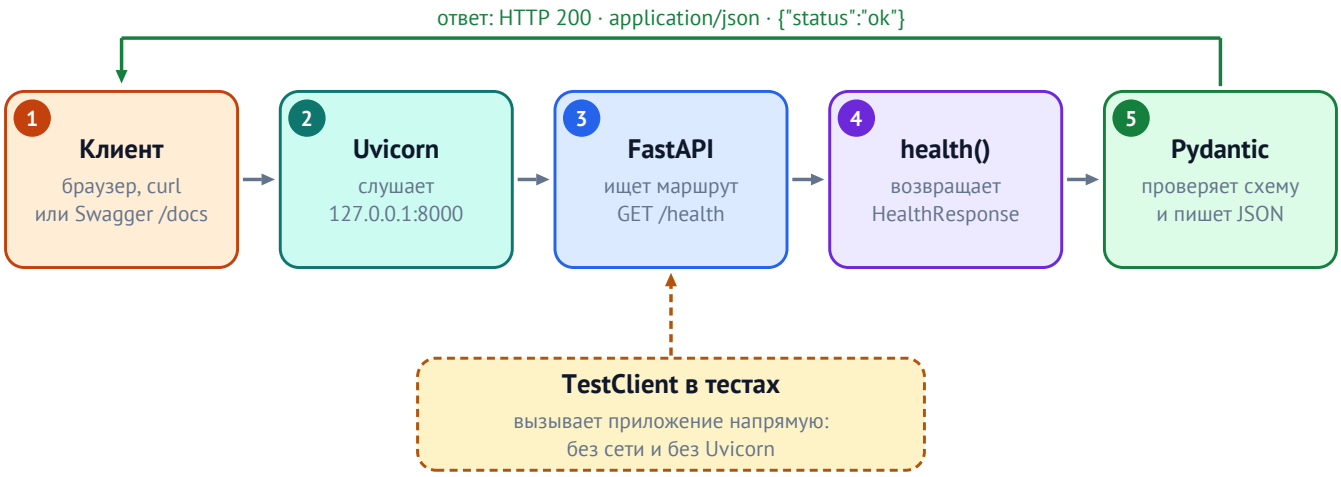
1 готово **Подготовка проекта**	2 готово **PostgreSQL и схема БД**	3 готово **Генерация данных**	4 готово **Аналитическое ядро**	5 готово **Поиск аномалий**
6 далее **REST API**	7 AI-агент	8 Интерфейс и отчёты	9 Тесты и безопасность	10 Docker и публикация

Числа считает код, а не LLM. Выручку, средний чек, сравнения и аномалии рассчитывает детерминированный SQL или Python. Модель понимает вопрос, выбирает инструменты и объясняет результат. Поэтому так важны правила в базе, точные типы денег и тесты.

Чего ещё нет: REST API метрик (06), LLM-агента (07) и интерфейса (08).

2 Путь запроса

Что происходит между нажатием Execute в /docs и ответом {"status": "ok"}.



Шаг	Кто	Что делает
1	Клиент	Формирует HTTP-запрос GET /health и отправляет его на 127.0.0.1:8000.
2	Uvicorn	Принимает соединение, разбирает HTTP и передаёт запрос приложению по протоколу ASGI.
3	FastAPI	Ищет обработчик по паре «метод + путь». Для POST ответит 405, для неизвестного пути — 404.
4	health()	Создаёт объект HealthResponse(status="ok").
5	Pydantic	Проверяет объект по схеме ответа и превращает его в JSON.

Фактический ответ сервера

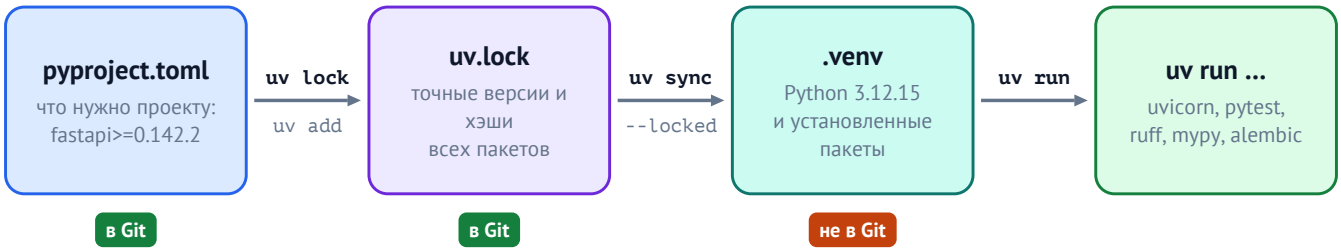
```
$ curl -i http://127.0.0.1:8000/health
HTTP/1.1 200 OK
server: uvicorn
content-length: 15
content-type: application/json

{"status":"ok"}
```

TestClient в тестах подключается сразу к FastAPI (шаг 3): без сети, без Uvicorn и без свободного порта. Поэтому unit-тесты проходят за доли секунды.

3 Окружение и uv

Как uv превращает список требований в одинаковое окружение на любом компьютере.



Виртуальное окружение `.venv` — отдельная папка со своим Python и библиотеками только для этого проекта. Так проекты не мешают друг другу. Системный `python3` на этом Mac — 3.9.6; его не трогаем, всё запускаем через `uv run`.

	pyproject.toml	uv.lock
Кто пишет	Человек и команда uv add	Только uv
Что внутри	Требования: fastapi>=0.142.2	Точные версии и хэши файлов
Транзитивные пакеты	Нет	Да, все 52: starlette, pydantic, anyio…
В Git	Да	Да

Зависимости проекта (версии из uv.lock)

Пакет	Версия	Группа	Зачем
`alembic`	`1.20.0`	runtime	миграции схемы
`fastapi`	`0.142.2`	runtime	маршруты, проверка данных, /docs
`psycopg`	`3.3.6`	runtime	драйвер PostgreSQL
`pydantic-settings`	`2.15.0`	runtime	настройки из окружения и .env
`scikit-learn`	`1.9.1`	runtime	
`sqlalchemy`	`2.1.3`	runtime	модели, пул, сессии
`uvicorn`	`0.54.0`	runtime	ASGI-сервер, слушает порт
`httpx2`	`2.13.1`	dev	HTTP-клиент для TestClient
`mypy`	`2.4.0`	dev	проверка типов
`pytest`	`9.1.1`	dev	запуск тестов
`ruff`	`0.16.10`	dev	линтер и форматировщик
`starlette`	`1.7.0`	fastapi	веб-основа FastAPI
`pydantic`	`2.13.5`	fastapi	модели и проверка данных

Политика возраста пакетов. `[tool.uv] exclude-newer = "2026-10-06T00:00:00Z"` — uv не видит выпуски PyPI новее этой даты. Свежий выпуск бывает сломан или даже вредоносен; за две недели такие проблемы обычно находят.

4 Создание проекта

Какими командами создан проект, чем они отличаются от стартового задания и что лежит в каждой папке.

```
# uv 0.12.23 — та же версия, что закреплена в CI
curl -LsSf https://astral.sh/uv/0.12.23/install.sh \
  | env UV_NO_MODIFY_PATH=1 sh

uv init --app --no-package --python 3.12 --author-from none
rm main.py && mkdir -p app tests
uv add fastapi 'uvicorn[standard]' pydantic-settings
uv add --dev pytest ruff mypy httpx2
```

uv init в uv 0.12

Без флагов создаёт пакет-библиотеку: src/ai_data_analyst/, [build-system] и [project.scripts]. Для веб-сервиса с папкой app/ в корне нужен --app --no-package.

httpx2 вместо httpx

Starlette 1.7 предупреждает: «Using httpx with starlette.testclient is deprecated; install httpx2 instead». TestClient работает на httpx2.

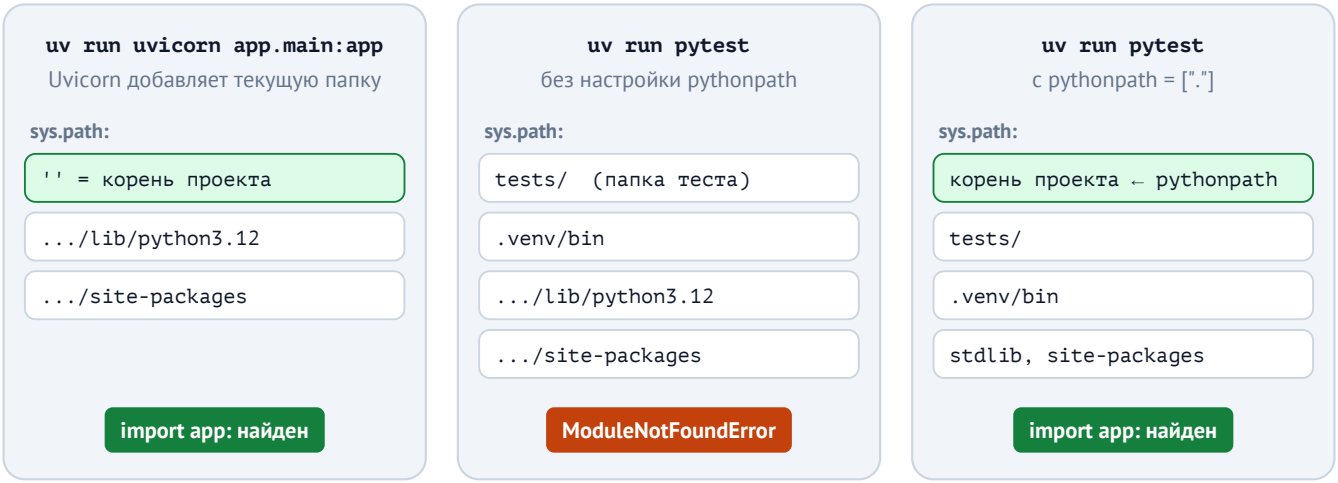
Файлы этапа 01

Файл или папка	Для чего нужен
app/__init__.py	Делает app пакетом; хранит версию __version__ = "0.1.0".
app/config.py	Класс Settings и функция get_settings().
app/main.py	Объект FastAPI и служебные маршруты.
tests/	Тесты: API, настройки, версия; позже — база.
pyproject.toml	Описание проекта, зависимости, настройки pytest, Ruff, mypy, uv.
uv.lock	Точные версии всех 52 пакетов.
.python-version	3.12 — какой Python выбирает uv.
.env.example	Образец локальных настроек; сам .env — не в Git.
.gitignore	Что Git не отслеживает: .venv, кеши, .env.
.github/workflows/ci.yml	Проверки в GitHub Actions.
docs/	Руководства, инструкции, CI/CD, подход, ADR, журнал, отчёт, PDF.
scripts/	Сборка этого PDF на русском и английском.
AGENTS.md	Правила работы над проектом.

5 Импорт и sys.path

Почему сервер находит пакет app, а pytest без настройки — нет.

Модуль — один файл `.py`. **Пакет** — папка с модулями; файл `__init__.py` выполняется при первом импорте пакета. **sys.path** — список папок, где Python ищет модули. Строка `from app.main import app` работает, только если в одной из этих папок лежит папка app.



Фактическая ошибка до настройки

```
$ uv run pytest
ImportError while importing test module '.../tests/test_health.py'.
E   ModuleNotFoundError: No module named 'app'
Interrupted: 3 errors during collection
```

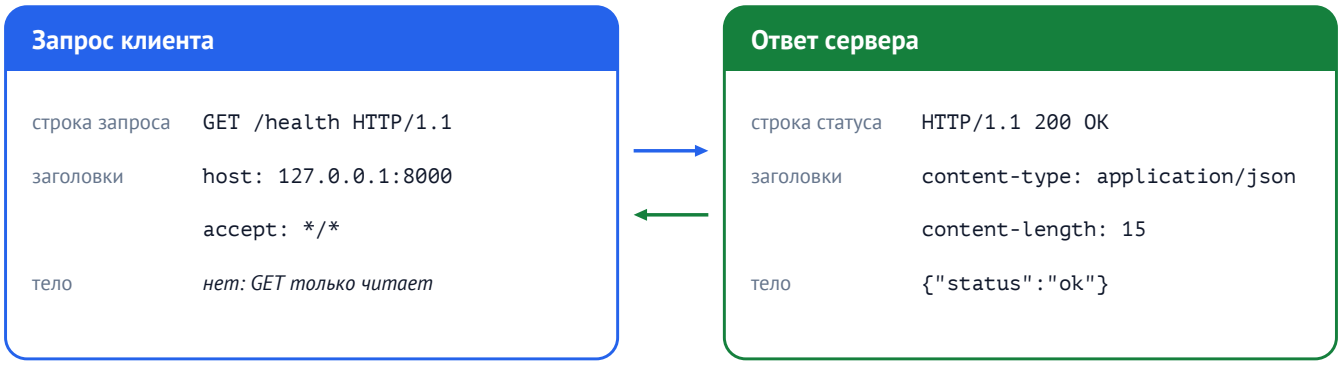
Исправление в pyproject.toml

```
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
```

Почему так. Проект создан с --no-package: в .venv ставятся только зависимости, сам app не устанавливается. Другие решения — python -m pytest (флаг -m добавляет текущую папку) или оформить проект пакетом. Явная настройка работает при любом способе запуска.

6 HTTP и JSON

Из чего состоят запрос и ответ и какие коды встретятся в проекте.



Метод	Смысл	В этом проекте
`GET`	Получить данные, ничего не меняя	GET /health, GET /ready; позже GET /api/metrics/revenue
`POST`	Создать объект или запустить действие	позже — вопрос AI-агенту
`PUT / PATCH`	Заменить или частично изменить	пока не используем
`DELETE`	Удалить	пока не используем

Код	Значение	Когда увидишь (проверено)
200 OK	Успех	GET /health → 200 OK
404 Not Found	Такого пути нет	GET /nope → 404 Not Found
405 Method Not Allowed	Путь есть, метод не тот	POST /health → 405 Method Not Allowed, allow: GET
422 Unprocessable Content	Неверные входные данные	появится на этапе 06 с параметрами запросов
500 Internal Server Error	Ошибка на сервере	эксперимент в разделе 7
503 Service Unavailable	Сервис не готов	GET /ready без базы — раздел 18

JSON — текстовый формат данных: объекты {...}, массивы [...], строки, числа, true / false / null. FastAPI превращает Pydantic-модель в JSON сам, а `response.json()` в тесте превращает JSON обратно в словарь Python.

7 Разбор кода API

Маршрут /health построчно и эксперимент с неправильным ответом.

```
app = FastAPI(
    title=get_settings().app_name,
    description="AI-powered business analytics platform for IndustrialFlow",
    version=__version__,
    lifespan=lifespan,
)


class HealthResponse(BaseModel):
    """Ответ проверки живости: процесс запущен и отвечает на HTTP-запросы."""

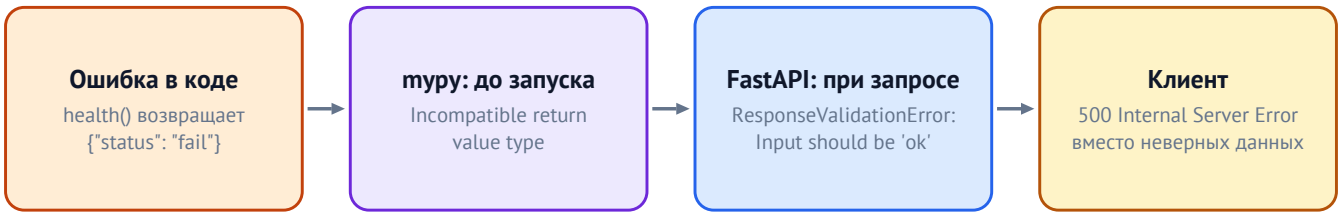
    # Literal допускает ровно одно значение: ответ с другим FastAPI не отправит.
    status: Literal["ok"]


@app.get("/health")
def health() -> HealthResponse:
    """Сообщить, что API работает; базу данных этот маршрут не проверяет."""
    return HealthResponse(status="ok")
```

Фрагмент	Что делает
`app = FastAPI(...)`	Объект приложения. Его находит Uvicorn (app.main:app) и вызывает TestClient. title и version — в /docs; lifespan закрывает пул базы при остановке.
`class HealthResponse`	Pydantic-модель: описание формы ответа. Literal["ok"] разрешает ровно одно значение.
`@app.get("/health")`	Регистрирует функцию для пары GET + /health. Другие методы получат 405.
`-> HealthResponse`	Аннотация возврата становится схемой ответа: по ней строится /docs и проверяется результат функции.
`def, не async def`	Обычную функцию FastAPI выполняет в отдельном потоке. К async вернёмся на этапе 06.

Эксперимент: неправильный ответ

Во временной копии проекта функция health() вернула `{"status": "fail"}`. Фактические результаты:



Два уровня защиты: типы ловят ошибку в коде, Pydantic — в данных. Неправильный ответ не уходит клиенту молча. Для аналитики это главное: лучше явная ошибка, чем неверное число в отчёте.

Модель → JSON → словарь

FastAPI превращает объект модели в JSON-текст, а тест превращает JSON обратно в словарь Python. Значения получены из настоящего приложения при сборке PDF:

```
>>> HealthResponse(status="ok").model_dump_json()   # model → JSON
'{"status":"ok"}'
>>> client.get("/health").json()                    # JSON → dict
{'status': 'ok'}
```

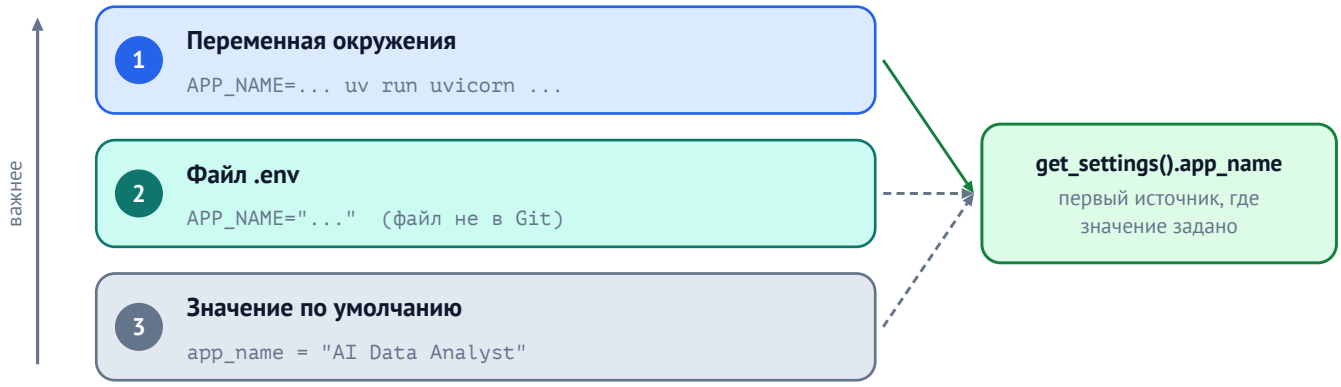
OpenAPI и страница /docs

FastAPI собирает описание маршрутов и моделей в документ OpenAPI (`GET /openapi.json`). Страница /docs — это Swagger UI, который рисует этот документ. Схема HealthResponse из приложения; docstring модели стал полем description:

```
{
  "properties": {
    "status": {
      "type": "string",
      "const": "ok",
      "title": "Status"
    }
  },
  "type": "object",
  "required": [
    "status"
  ],
  "title": "HealthResponse",
  "description": "Ответ проверки живости: процесс запущен и отвечает на HTTP-запросы."
}
```

8 Настройки

Конфигурация живёт в окружении, а не в коде.




```
class Settings(BaseSettings):
    """Конфигурация, которая читается при создании объекта настроек.

    Приоритет источников: переменная окружения → строка в .env → значение
    по умолчанию ниже. Регистр имён не важен: поле app_name читается из
    переменной APP_NAME.
    """

    # extra="ignore": в .env лежат и ключи Docker (POSTGRES_*), которые
    # к этим настройкам не относятся; без него Pydantic отклонил бы их.
    # hide_input_in_errors: адрес БД содержит пароль — его нельзя выводить
    # в тексте ошибки проверки.
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
        hide_input_in_errors=True,
    )

    # Название сервиса в OpenAPI-схеме и на странице /docs.
    app_name: str = "AI Data Analyst"
    ...


@lru_cache
def get_settings() -> Settings:
    """Прочитать настройки один раз на процесс и дальше отдавать тот же объект.

    Функция вместо глобального объекта: настройки читаются при первом
    обращении, а тесты могут сбросить кеш через get_settings.cache_clear().
    """
    return Settings()
```

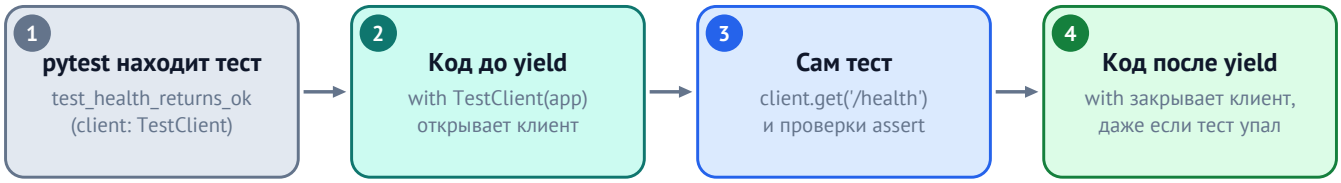
Тест	Что закрепляет
`test_default_app_name`	Без окружения и .env берётся значение из кода.
`test_env_file_overrides_default`	Строка в .env заменяет значение по умолчанию.
`test_environment_variable_beats_env_file`	Переменная окружения важнее строки в .env.

extra="ignore" — в .env лежат и переменные Docker (POSTGRES_*), не относящиеся к Settings. **get_settings()** читает настройки один раз на процесс, поэтому изменения .env видны после перезапуска сервера. На этапе 02 добавлен database_url — раздел 12.

9 Тесты API

Как pytest находит тесты, что такое фикстура и что доказывает каждый тест этапа 01.

Pytest находит файлы `test_*.py` в папке tests/ и выполняет функции с именем `test_...`. Внутри обычный assert: если условие ложно, pytest покажет, какие значения не совпали. Фикстура — подготовка для теста; тест получает её, объявив параметр с тем же именем.




```
@pytest.fixture
def client() -> Iterator[TestClient]:
    """Открыть тестовый клиент: запросы идут прямо в приложение, без сети.

    Фикстура выполняется перед каждым тестом, который принимает параметр
    client, а код после yield закрывает клиент после теста.
    """
    with TestClient(app) as test_client:
        yield test_client
```

Тест	Что доказывает
`test_health_returns_ok`	GET /health → 200 и ровно {"status": "ok"}
`test_health_rejects_post`	POST /health → 405
`test_openapi_describes_health`	Схема знает версию, путь и поле status
`test_default_app_name`	Умолчание из кода
`test_env_file_overrides_default`	.env важнее умолчания
`test_environment_variable_beats_env_file`	Окружение важнее .env
`test_version_matches_pyproject`	app.__version__ совпадает с pyproject.toml

> Это 7 тестов этапа 01. Сейчас в проекте 136 unit-тестов и 87 интеграционных. Проверки этапов 02–05 — в разделах 20, 23, 26 и 29.

10 Ruff и mypy

Два инструмента, которые находят ошибки до запуска программы.

Правило Ruff	Что ищет
E	Нарушения стиля PEP 8
F	Ошибки: неиспользуемые импорты, неопределённые имена
I	Порядок импортов
UP	Устаревший синтаксис, который в Python 3.12 проще
B	Типичные ловушки, например изменяемое значение аргумента по умолчанию

`ruff check .` ищет проблемы, с `--fix` исправляет безопасные; `ruff format --check .` проверяет формат, в том числе Python-блоков в Markdown. **mypy strict** требует аннотации у каждой функции; теперь он проверяет и `migrations/`.

Пример: что находит Ruff

```
from app.config import settings
import os
from typing import List


def names(items: List[str], seen=[]) -> List[str]:
    return [item for item in items if item not in seen]
```

```
$ uv run ruff check --output-format concise example.py
example.py:1:1: I001 [*] Import block is un-sorted or un-formatted
example.py:1:24: F401 [*] `app.config.settings` imported but unused
example.py:2:8: F401 [*] `os` imported but unused
example.py:3:1: UP035 `typing.List` is deprecated, use `list` instead
example.py:6:18: UP006 [*] Use `list` instead of `List` for type annotation
example.py:6:34: B006 Do not use mutable data structures for argument defaults
example.py:6:41: UP006 [*] Use `list` instead of `List` for type annotation
Found 7 errors.
[*] 5 fixable with the `--fix` option
    (1 hidden fix can be enabled with the `--unsafe-fixes` option).
```

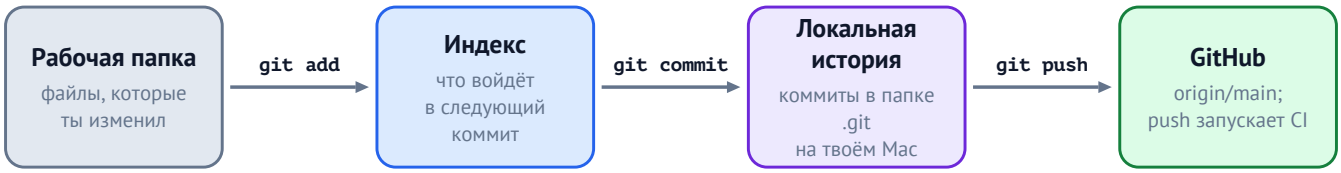
I001 — порядок импортов, F401 — неиспользуемый импорт, UP035 и UP006 — List вместо list, B006 — общий список по умолчанию, который копит значения между вызовами. Пометка [*] — Ruff исправит сам через --fix.

Пример: что находит mypy

```
$ uv run mypy app
app/main.py:30: error: Incompatible return value type
    (got "dict[str, str]", expected "HealthResponse")  [return-value]
Found 1 error in 1 file (checked 3 source files)
```

11 Git и GitHub

Что хранится в репозитории и как изменения попадают на GitHub.



В Git

Код app/, tests/, migrations/, sql/, pyproject.toml, uv.lock, compose.yaml, alembic.ini, .env.example, workflow, документация и PDF.

Не в Git

.venv (пересоздаётся uv sync), кеши инструментов, .env с паролем базы и будущими ключами, данные PostgreSQL в томе Docker.

Публикация

Репозиторий: **merkertg/ai-data-analyst**, ветка main. Для нового проекта: на github.com **New repository** без README, .gitignore и лицензии, затем:

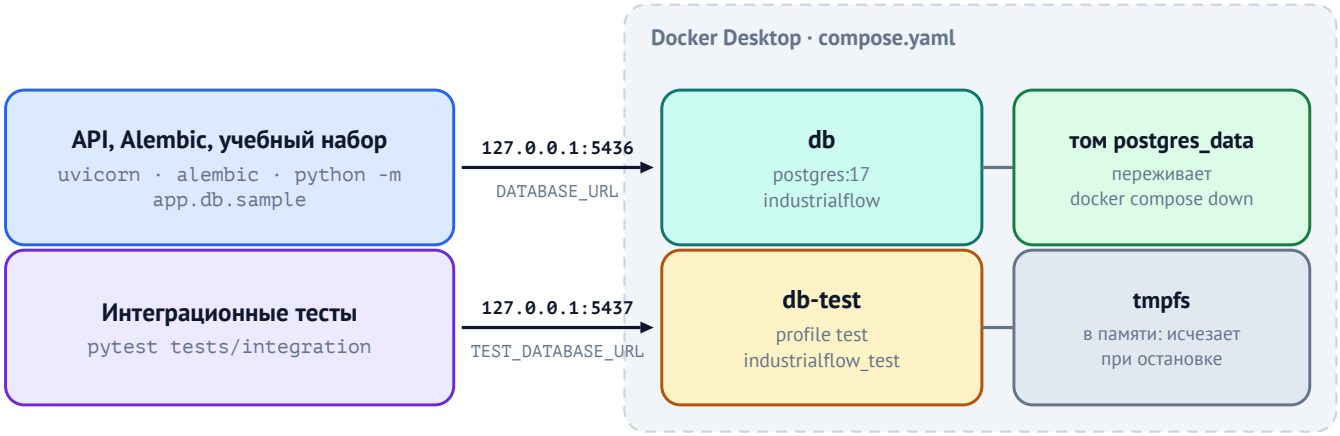
```
git remote add origin git@github.com:<login>/<repository>.git
git push -u origin main
```

Дальше каждое изменение: `git add` → `git commit` → `git push origin main`. Вкладка **Actions** показывает результат CI; только он подтверждает удалённую проверку.

> Сообщения коммитов — на английском, в повелительном наклонении: «Add …», «Fix …». Каждый коммит — законченный шаг, после которого проходят все проверки.

12 PostgreSQL в Docker

Где живёт база, как к ней подключаются приложение и тесты и что переживает перезапуск.



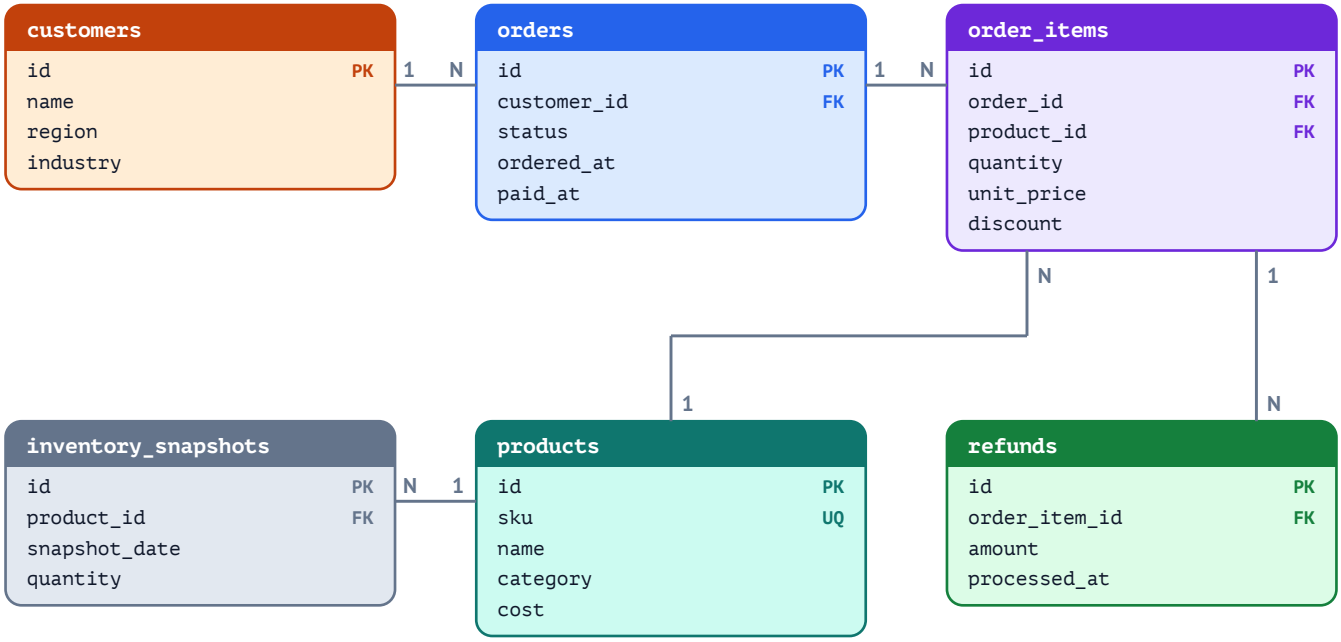

```
cp .env.example .env
docker compose up --detach --wait db
uv run alembic upgrade head
uv run python -m app.db.sample
docker compose exec db psql -U analyst -d industrialflow
docker compose stop
```

Решение	Зачем
Порт только на 127.0.0.1	База недоступна из локальной сети.
Порты 5436 / 5437	Не конфликтуют с другими PostgreSQL на Mac (первый проект занимает 5433/5434).
Образ по digest	Тег postgres:17 могут перезаписать; digest — нет. Тот же digest в CI.
healthcheck · pg_isready	--wait ждёт, пока база действительно примет соединения.
Пароль из .env	Без .env Compose остановится с подсказкой; пароль не попадает в Git.

Проверено: после docker compose down и нового up контейнер сменился, а 5 заказов и версия схемы 0001 остались в томе. **docker compose down -v удаляет том вместе с данными.**

13 Таблицы и связи

Шесть таблиц, первичные и внешние ключи, связи «один ко многим».



PK — первичный ключ, FK — внешний ключ, UQ — уникальное значение

Таблица	Что хранит
customers	Клиенты — промышленные предприятия: регион и отрасль.
products	Каталог: уникальный артикул, категория, себестоимость.
orders	Заказ: клиент, статус, когда оформлен и когда оплачен.
order_items	Позиции: товар, количество, цена продажи, скидка.
refunds	Возвраты денег по позиции и дата проведения.
inventory_snapshots	Остаток товара на конец дня.

Нормализация. Один заказ — много товаров, поэтому заказ и позиции лежат в разных таблицах: клиент и дата оплаты хранятся один раз в orders, регион — один раз в customers. Цена продажи — в позиции, а не в товаре: в истории должна остаться цена, по которой продали.

14 Правила целостности

28 именованных ограничений и 7 индексов: некорректную строку отвергнет сама PostgreSQL.

Вид	Сколько	Примеры в проекте
CHECK	14	quantity > 0; 0 ⩽ discount < 1; cost ⩾ 0; amount > 0; у оплаченного заказа есть paid_at, у остальных нет; paid_at ⩾ ordered_at
UNIQUE	3	артикул sku; один товар — одна строка заказа; один снимок в день
FOREIGN KEY	5	нельзя сослаться на несуществующего клиента, товар, заказ
PRIMARY KEY	6	id каждой таблицы
NOT NULL	—	все столбцы, кроме paid_at

Связь	При удалении	Почему
заказ → позиции	CASCADE	Позиции без заказа не имеют смысла.
клиент → заказы	RESTRICT	Клиента с историей продаж удалить нельзя.
товар → позиции, остатки	RESTRICT	Проданный товар остаётся в каталоге.
позиция → возвраты	RESTRICT	Проведённые деньги не исчезают; каскад удаления заказа упирается в возврат.

Фактические ошибки PostgreSQL

```
ERROR:  new row for relation "order_items" violates check constraint
        "ck_order_items_quantity_positive"
DETAIL:  Failing row contains (7, 1, 3, 0, 100.00, 0.0000).

ERROR:  update or delete on table "products" violates foreign key constraint
        "fk_inventory_snapshots_product_id_products" on table "inventory_snapshots"
DETAIL:  Key (id)=(1) is still referenced from table "inventory_snapshots".
```

Имена ограничений

Соглашение в app/db/base.py: ck_, fk_, uq_, ix_ + таблица + столбцы. По имени из ошибки видно правило; тесты проверяют именно имя.

Чего база не проверяет

Правила между таблицами: сумма возвратов не больше позиции, возврат только по оплаченному заказу. Их проверяет контроль качества этапа 03.

15 Деньги и время

Почему деньги — NUMERIC и Decimal, а время — timestamptz и московский месяц.

```
>>> 0.1 + 0.2
0.30000000000000004
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2")
Decimal('0.3')
```

Где	Тип	Смысл
Цена, себестоимость, возврат	NUMERIC(12, 2)	до 9 999 999 999.99 ₽ с точностью до копейки
Скидка	NUMERIC(5, 4)	доля: 0.1250 = 12.5 %
Python	Decimal	SQLAlchemy читает NUMERIC как Decimal
Сумма позиции	ROUND(q × price × (1 − d), 2)	округление каждой позиции до копейки

Слишком большое значение база отвергнет ошибкой numeric field overflow, а не исказит — это проверяет тест test_money_overflow_is_rejected.

Один момент — два месяца



timestamptz хранит **момент**, а не «часы на стене». Заказ №5 оплачен 1 октября в 01:30 по Москве = 30 сентября 22:30 UTC. Отчётный месяц компании — московский, поэтому запросы используют `AT TIME ZONE 'Europe/Moscow'`. По UTC сентябрь дал бы 328 000.00 вместо 310 000.00 — это проверяет тест test_month_depends_on_time_zone.

16 SQLAlchemy и сессии

Модели-классы описывают таблицы; пул и сессии управляют соединениями и транзакциями.

```
class Order(Base):
    """Заказ клиента. Оплата фиксируется временем paid_at."""

    __tablename__ = "orders"
    __table_args__ = (
        CheckConstraint(
            "status IN ('pending', 'paid', 'cancelled')", name="status_allowed"
        ),
        CheckConstraint(
            "(status = 'paid') = (paid_at IS NOT NULL)", name="paid_at_matches_status"
        ),
        CheckConstraint("paid_at >= ordered_at", name="paid_after_ordered"),
        Index(None, "customer_id"),
        Index(None, "paid_at"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    customer_id: Mapped[int] = mapped_column(
        ForeignKey("customers.id", ondelete="RESTRICT")
    )
    status: Mapped[str] = mapped_column(
        String(16), default=OrderStatus.PENDING, server_default=OrderStatus.PENDING
    )
    ordered_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
    paid_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))

    customer: Mapped[Customer] = relationship(back_populates="orders")
    items: Mapped[list["OrderItem"]] = relationship(
        back_populates="order", cascade="all, delete-orphan", passive_deletes=True
    )
```

Элемент	Что означает
`Mapped[int]`	Тип для Python и mypy; без None — NOT NULL.
`Mapped[datetime | None]`	Столбец может быть пустым (NULL).
`mapped_column(...)`	Тип в базе и ограничения столбца.
`__table_args__`	Ограничения и индексы всей таблицы.
`relationship(...)`	Связь объектов: order.customer, order.items. Это не столбец.

Пул соединений и сессии

Engine

Пул соединений. Создаётся лениво: импорт и /health базу не трогают. pool_pre_ping отбрасывает разорванные соединения.

Session

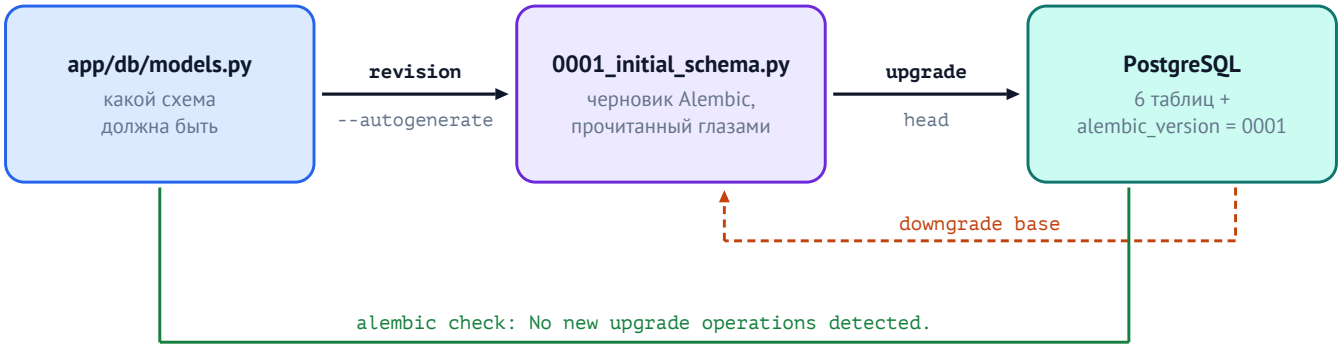
Рабочая область: flush() отправляет строки, commit() фиксирует, rollback() отменяет всё в транзакции.

Транзакция

Всё или ничего. Скрытого commit нет: код, который пишет данные, фиксирует их сам.

17 Миграции Alembic

Схема меняется только миграциями: история, откат и одинаковая база у всех.




```
$ uv run alembic upgrade head
INFO  [alembic.runtime.migration] Running upgrade  -> 0001, Initial schema: ...
$ uv run alembic current
0001 (head)
$ uv run alembic check
No new upgrade operations detected.
```

Команда	Что делает
`alembic upgrade head`	Применить все неприменённые миграции
`alembic downgrade -1`	Откатить одну миграцию
`alembic downgrade base`	Откатить всё — с удалением данных
`alembic current`	Версия в базе
`alembic history`	Список миграций
`alembic check`	Есть ли изменения моделей без миграции
`alembic upgrade head --sql`	Показать SQL, не выполняя

Почему не create_all(). Он только создаёт отсутствующие таблицы и не меняет существующие. **Автогенерация — черновик**: файл читают глазами. **Применённую миграцию не редактируют** — любое изменение схемы оформляют новой миграцией 0002, 0003…

18 /health и /ready

Жив ли процесс и можно ли отправлять ему запросы — два разных вопроса.



	/health	/ready
Вопрос	Процесс жив?	Можно принимать запросы?
Обращается к базе	Нет	Да: соединение и версия схемы
Без базы	200	503
Кому нужен	перезапуск упавшего процесса	решение, отправлять ли трафик

Фактическая проверка на настоящем сервере

```
# база работает / database running
HTTP/1.1 200 OK                   {"status":"ready","revision":"0001"}
# docker compose stop db
HTTP/1.1 503 Service Unavailable  {"detail":"Database is not ready"}
# docker compose start db
HTTP/1.1 200 OK                   {"status":"ready","revision":"0001"}
```

Третий ответ показывает pool_pre_ping: пул выбросил разорванные соединения и открыл новые без перезапуска API. Ответ 503 одинаков для всех причин — адрес базы и ошибки драйвера наружу не выходят.

19 SQL вручную

Четыре запроса на учебном наборе, все ответы посчитаны заранее.

Заказ	Статус	Состав	Сумма
1	оплачен 5 сен	2 насоса × 100 000 − 10 %, 5 датчиков × 2 000	190 000.00
2	оплачен 20 сен	1 насос × 120 000	120 000.00
3	отменён	10 кранов × 4 500	—
4	не оплачен	3 датчика × 2 000	—
5	оплачен 1 окт 01:30 МСК	4 крана × 4 500	18 000.00

Возвраты по датчикам заказа 1: 1 000.00 (10 сентября) и 500.00 (2 октября). Запуск файла: `docker compose exec -T db psql -U analyst -d industrialflow < sql/02_net_revenue_by_month.sql`

01 · Суммы оплаченных заказов: JOIN и GROUP BY

```
 order_id |        customer        |        paid_at         | items | order_total
----------+------------------------+------------------------+-------+-------------
        1 | ООО «ТехМонтаж-Север»  | 2026-09-05 09:00:00+00 |     2 |   190000.00
        2 | АО «Южный химкомбинат» | 2026-09-20 06:30:00+00 |     1 |   120000.00
        5 | ООО «ТехМонтаж-Север»  | 2026-09-30 22:30:00+00 |     1 |    18000.00
```

psql показывает время в UTC (+00): 09:00 UTC = 12:00 по Москве.

02 · Чистая выручка по месяцам: CTE и FULL OUTER JOIN

```
WITH sales AS (
    SELECT
        date_trunc('month', o.paid_at AT TIME ZONE 'Europe/Moscow')::date AS month,
        SUM(ROUND(oi.quantity * oi.unit_price * (1 - oi.discount), 2)) AS paid_sales
    FROM orders AS o
    JOIN order_items AS oi ON oi.order_id = o.id
    WHERE o.status = 'paid'
    GROUP BY 1
),
refunded AS (
    SELECT
        date_trunc('month', r.processed_at AT TIME ZONE 'Europe/Moscow')::date AS month,
        SUM(r.amount) AS refunds
    FROM refunds AS r
    GROUP BY 1
)
SELECT
    COALESCE(s.month, r.month) AS month,
    COALESCE(s.paid_sales, 0) AS paid_sales,
    COALESCE(r.refunds, 0) AS refunds,
    COALESCE(s.paid_sales, 0) - COALESCE(r.refunds, 0) AS net_revenue
FROM sales AS s
FULL OUTER JOIN refunded AS r ON r.month = s.month
ORDER BY month;
```

```
   month    | paid_sales | refunds | net_revenue
------------+------------+---------+-------------
 2026-09-01 |  310000.00 | 1000.00 |   309000.00
 2026-10-01 |   18000.00 |  500.00 |    17500.00
```

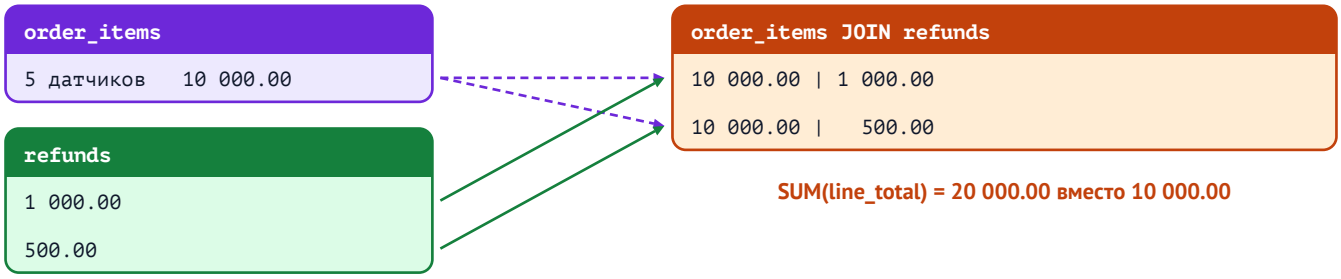
Правило курса: Net Revenue = Paid Sales − Processed Refunds; возврат уменьшает выручку месяца, в котором проведён.

03 · Продажи по категориям: оконная функция

```
     category      | paid_sales | share_percent
-------------------+------------+---------------
 Насосы            |  300000.00 |          91.5
 Запорная арматура |   18000.00 |           5.5
 Датчики           |   10000.00 |           3.0
```

SUM(...) OVER () видит итог по всем строкам результата, не схлопывая их, — так считается доля категории.

04 · Ловушка двойного подсчёта




```
 naive_sales | correct_sales | refunds
-------------+---------------+---------
   338000.00 |     328000.00 | 1500.00
```

JOIN «один ко многим» повторяет строку позиции на каждый её возврат: 338 000.00 вместо 328 000.00. Правило: сначала свести таблицу «многих» к одной строке на ключ (или считать суммы отдельно, как в запросе 02), потом соединять.

20 Тесты с базой

136 unit-тестов без базы и 87 интеграционных на настоящей PostgreSQL.



Файл этапов 01–02	Тестов	Что проверяет
test_config.py	10	приоритет настроек; драйвер и имя базы; пароль скрыт
test_ready.py	4	/ready без базы и с недоступной базой → 503
test_session.py	3	ленивый пул, DatabaseNotConfigured
test_models.py	5	6 таблиц, NUMERIC(12,2), правила удаления, имена
integration/test_migrations.py	5	upgrade, downgrade, alembic check, /ready
integration/test_constraints.py	23	20 плохих строк, каскад, Decimal, timestamptz
integration/test_queries.py	6	4 SQL-файла и часовой пояс на учебном наборе

```
@pytest.mark.parametrize("case", CASES, ids=list(CASES))
def test_invalid_row_is_rejected(db_session: Session, case: str) -> None:
    """Плохая строка вызывает IntegrityError с именем нужного ограничения."""
    build, constraint = CASES[case]
    db_session.add(build(base_rows(db_session)))

    with pytest.raises(IntegrityError) as error:
        db_session.flush()

    assert violated_constraint(error.value) == constraint
```

```
docker compose --profile test up --detach --wait db-test
export TEST_DATABASE_URL=\
  'postgresql+psycopg://analyst:analyst_test@127.0.0.1:5437/industrialflow_test'
uv run pytest
uv run pytest -m integration
```

Адрес — только из TEST_DATABASE_URL и только база *_test: тесты сбрасывают схему и не должны задеть базу разработки. Без адреса локально — skip интеграционных тестов, в CI (CI=true) — ошибка.

21 Воспроизводимые данные

Один seed и те же параметры дают одинаковые файлы без подключения к базе.

Новый набор для IndustrialFlow: **20 000 заказов, 640 клиентов, 64 товара, 8 регионов**, январь–сентябрь 2026 года. Компания и операции вымышлены. Месяцы считаются по Europe/Moscow.

```
uv run python -m app.data.generate \
  --output data/demo --seed 42 --orders 20000
```

> **Каталог должен быть новым или пустым.** Генератор не перезаписывает существующий набор. Для повторного запуска выбери другой путь, например data/demo-repeat.

Что появляется в каталоге

Файл	Содержание
`customers.csv, products.csv`	Клиенты и каталог товаров.
`orders.csv, order_items.csv`	Заказы и позиции заказов.
`refunds.csv`	Проведённые возвраты денег.
`inventory_snapshots.csv`	Ежедневные остатки товаров.
`manifest.json`	Версия формата, seed, число строк и контрольные суммы CSV.
`ground_truth.json`	Исходные и изменённые показатели сценариев, только для оценки.

Почему результат повторяется

Свой random.Random(seed) задаёт последовательность выборок. Decimal считает деньги, CSV хранит их десятичным текстом. NumPy и Faker не нужны. Те же параметры и версия генератора дают одинаковые байты; другой seed меняет конкретные заказы.

```
uv run python -m app.data.generate \
  --output data/demo-repeat --seed 42 --orders 20000
diff -r data/demo data/demo-repeat
```

diff без различий и с кодом 0 подтверждает совпадение. Сгенерированные наборы не включаются в Git.

22 Сценарии A01–A05

Необычное деловое событие может полностью соблюдать правила данных.

Генератор сначала создаёт исходные данные, затем внедряет пять сценариев. Это **валидные операции**, которые должны проходить проверку качества. Позднее аналитика должна обнаружить их проявления в таблицах.

ID	Когда и где	Изменение
A01	Сентябрь, оплаченные насосы	Количество = 40% исходного, округление вниз, минимум 1.
A02	Август, Юг, оплаченная арматура	Скидка 35%.
A03	Июль, оплаченные компрессоры	Дополнительный возврат 30% округлённой суммы позиции.
A04	10–19 сентября, насос с ID 1	Остаток равен нулю.
A05	Одна оплаченная позиция датчиков в июне	Количество равно 500.

A01 и A04 не доказывают причинность. Снижение количества насосов и нулевой остаток одного товара совпадают по времени. Это повод проверить гипотезу, а не готовый вывод о причине.

Что такое ответ для оценки

ground_truth.json хранит исходные и наблюдаемые величины, вычисленные кодом. Он нужен тестам и будущей оценке аналитики. **Будущий LLM-агент его не читает:** агент должен получать числа из SQL/Python-инструментов поверх базы.

Различай два вопроса. «Возвраты больше стоимости позиции?» — проверка корректности. «Почему возвратов стало больше?» — аналитический вопрос. Успешная проверка качества не исключает необычных событий бизнеса.

23 Загрузка и качество

Одна транзакция для импорта; отдельная команда только читает базу.

```
docker compose up --detach --wait db
uv run alembic upgrade head
uv run python -m app.data.load --input data/demo
uv run python -m app.data.quality
```

Все шесть таблиц должны быть пустыми. Маленький набор app.db.sample и полный demo выбираются до загрузки. При существующих строках загрузчик отказывает; параметра сброса нет.

Что защищает загрузку

Проверка комплекта и манифеста выявляет неполные или изменённые файлы. Клиенты и товары загружаются до ссылающихся на них строк. Одна транзакция фиксирует весь набор; при ошибке импорт откатывается целиком. Обычные команды берут DATABASE_URL из настроек проекта.

Что проверяет quality

Правило	Пример нарушения
В каждом заказе есть позиции	Есть заказ, у которого нет ни одной позиции.
Сумма возвратов ⩽ сумма позиции	Позиция 100.00, два возврата по 60.00 = 120.00.
Возврат только по оплате	Возврат у pending или cancelled.
Возврат не раньше оплаты	Оплата 12 июля, возврат 11 июля.

Сумма позиции = ROUND(quantity × unit_price × (1 − discount), 2). quality не меняет строки: код 0 — правила прошли, 1 — найдены нарушения. Ограничения PostgreSQL продолжают защищать отдельные строки, внешние ключи и уникальность.

Интеграционные тесты: только TEST_DATABASE_URL и база с суффиксом _test. Базу разработки из .env тесты не используют. Фактические результаты — в разделе 34 и docs/VERIFICATION.md; удалённый GitHub Actions подтверждается отдельно.

24 Точные показатели

SQL считает деньги; Python Decimal сохраняет точность результата.

Этап 04 добавляет **app/analytics/** и JSON CLI без новых зависимостей. PostgreSQL и Decimal вычисляют показатели. LLM не участвует; Pandas и DuckDB отложены по ADR 0004.

Поле	Правило
`paid_sales`	Продажи оплаченных заказов по paid_at.
`refunds`	Возвраты по processed_at, только по оплаченным заказам.
`net_revenue`	`paid_sales − refunds`
`paid_orders`	Каждый оплаченный заказ один раз.
`average_order_value`	Продажи / заказы, до возвратов; null без оплат.

Период и округление

[start, end) — даты в Europe/Moscow: начало включено, конец исключён. Сентябрь: 2026-09-01 → 2026-10-01. Каждая позиция округляется до копеек **до суммирования**: ROUND(quantity × unit_price × (1 − discount), 2). Деньги — NUMERIC/Decimal, в рублях без НДС.

> **Продажи и возвраты агрегируются отдельно.** Несколько возвратов не повторяют продажу. Заказы считаются из orders, а не из строк JOIN. Октябрьский возврат сентябрьской покупки уменьшает октябрьскую выручку.

```
uv run python -m app.analytics \
  --start 2026-09-01 --end 2026-10-01
```

JSON: суммы строками, неопределённые значения null. Все запросы CLI видят один снимок в транзакции REPEATABLE READ READ ONLY. DATABASE_URL берётся из настроек; сервер API не нужен.

25 Месяцы и сравнения

Пустые месяцы остаются в ряду; нулевой знаменатель не даёт процент.

monthly_metrics строит календарные отрезки, включая пустые месяцы. Частичные первый и последний месяцы обрезаны границами запроса; максимум 120 отрезков. Нет оплат — средний чек null. Только возвраты — выручка может быть отрицательной.

Ручной пример: app.db.sample

Показатель	Сентябрь	Октябрь
Продажи	310000.00	18000.00
Возвраты	1000.00	500.00
Чистая выручка	309000.00	17500.00
Оплаченные заказы	2	1
Средний чек	155000.00	18000.00

Это маленький набор этапа 02, **не demo из 20 000 заказов**. В октябре есть возврат 500.00 по сентябрьской продаже.

```
uv run python -m app.analytics \
  --start 2026-10-01 --end 2026-11-01 \
  --compare-start 2026-09-01 --compare-end 2026-10-01 \
  --dimension category
```

Разница и процент

Разница = current − previous. Процент = разница / abs(previous) × 100, округление ROUND_HALF_UP до двух знаков. Выручка: 17500 − 309000 = **−291500.00 (−94.34%)**. Прошлый показатель −100, текущий −50: улучшение +50%.

> **previous = 0 → процент null.** Если среднего чека нет хотя бы в одном периоде, его сравнение тоже null. Периоды сравниваются по суммам: разная длина не пересчитывается в показатель «на день».

26 Вклады групп

Вклад объясняет арифметику изменения, а не причину поведения покупателей.

Измерения из разрешённого списка: **category** — категория товара, **region** — регион клиента. Группы берутся из обоих периодов, включая группы только с возвратами. Сортировка: разница по возрастанию, затем название.

> **Вклад, % = разница выручки группы / общая разница × 100.** Знаки сохраняются. Нулевая общая разница даёт null для всех долей.

Октябрь к сентябрю: учебный набор

Категория	Разница, RUB	Вклад
Насосы	−300000.00	102.92%
Датчики	−9500.00	3.26%
Запорная арматура	+18000.00	−6.17%
Всего	−291500.00	100.01%

Насосы дают больше 100% падения: арматура частично его компенсирует и имеет отрицательный вклад в общее снижение. Датчики остаются в октябре только с возвратом. После округления сумма долей 100.01%, а не ровно 100% — это допустимо.

```
uv run pytest tests/test_analytics_models.py \
  tests/test_analytics_cli.py
uv run pytest tests/integration/test_analytics.py
```

Интеграционные тесты — только TEST_DATABASE_URL с суффиксом _test. Проверяются ручные суммы, двойной счёт, округление, Московские границы, пустые месяцы, сравнения и вклады. Фактические результаты — раздел 34 и docs/VERIFICATION.md.

> **Следующие этапы:** поиск аномалий — 05; REST API метрик — 06; LLM-агент — 07. Изменение выручки по группе само по себе не доказывает причину падения.

Полный набор seed 42: сентябрь к августу

Тот же CLI на 20 000 заказах этапа 03, загруженных в отдельную пустую базу *_test. Фактический вывод 21 октября 2026 года:

```
uv run python -m app.analytics \
  --start 2026-09-01 --end 2026-10-01 \
  --compare-start 2026-08-01 --compare-end 2026-09-01 \
  --dimension category
```

Показатель	Август	Сентябрь	Изменение
Продажи	633 733 255.24	468 349 280.42	−165 383 974.82 (−26.10%)
Возвраты	1 557 126.03	1 670 523.07	+113 397.04 (+7.28%)
Чистая выручка	632 176 129.21	466 678 757.35	−165 497 371.86 (−26.18%)
Оплаченные заказы	1 985	1 931	−54 (−2.72%)
Средний чек	319 261.09	242 542.35	−76 718.74 (−24.03%)

Категория	Разница выручки	Вклад
pumps	−138 597 069.79	83.75%
compressors	−26 317 759.86	15.90%
sensors	−294 972.94	0.18%
valves	−287 569.27	0.17%

Это сценарий A01. Заказов почти столько же (−2.72%), а средний чек упал на 24%: насосов покупают меньше. Насосы дают 83.75% падения выручки. Аналитика нашла это, не читая ground_truth.json. По регионам падение распределено ровно: от 7.72% (Ural) до 18.56% (Central).

Помесячный ряд январь–сентябрь показывает и сценарий **A03**: возвраты июля 132 586 941.37 при 0.8–1.7 млн в другие месяцы. Это измерение, а не объяснение: почему упали насосы (склад, цена, спрос), из этих чисел не следует. Автоматический поиск отклонений — этап 05.

27 Статистические кандидаты

Необычное наблюдение — повод проверить данные и гипотезы.

Этап 05 читает текущую базу и выдаёт **кандидатов**. Правила не знают сценарии A01–A05. Деньги считает SQL/Decimal; LLM не участвует. Параметры ниже — открытые учебные настройки.

Метод	Что и как проверяется
Медиана и MAD	Месячное количество и доля возвратов по категории; доля скидки по региону и категории. Не менее 6 других месяцев того же ряда; $\|z\| \geqslant 6$.
MAD = 0	Разница строго больше max(25% от |медианы|, floor). floor = 1 для количества, 0.01 для долей.
IQR	Количество оплаченной позиции выше Q3 + 3 × IQR в своей категории; не менее 8 позиций.
Нулевой остаток	Максимальный отрезок из $\geqslant 3$ дней подряд с остатком 0. Пропущенный день разрывает отрезок.

Месяц не задаёт себе ориентир

Медиана — середина ряда; MAD — медиана абсолютных отклонений от неё. Проверяемый месяц исключён из ориентира (leave-one-out): $z = 0.6745 \times (x - \text{median}) / \text{MAD}$. Пример: median = 100, MAD = 2, x = 60 → z = −13.49, кандидат по правилу $|z| \geqslant 6$.

> **Это ретроспектива.** Другие месяцы могут идти после проверяемого. Месячные детекторы берут только полные месяцы: обрезанные граничные месяцы пропускаются. Короткая история может не дать оценки. Прогноз не строится.

28 Isolation Forest

Модель ищет сочетания признаков; финансовые суммы остаются точными.

Isolation Forest из scikit-learn дополняет простые правила. Одна строка — полный месяц, регион и категория. Нужно хотя бы 16 пригодных строк; полностью пустые группы не заменяются нулями.

Признак	Смысл
Количество / медиана	Относительно своего ряда.
Продажи / медиана	Отношение точных сумм.
Доля скидки	Взвешена по исходной стоимости.
Возвраты / продажи	Без продаж доля не определена.

SQL независимо агрегирует продажи и возвраты. Денежные суммы остаются NUMERIC/Decimal. Только рассчитанные **безразмерные отношения** становятся float для модели.

Воспроизводимый запуск

100 деревьев; random_state = 42; n_jobs = 1; contamination = auto. Кандидаты имеют decision_function < 0. Оценка — не вероятность и не сумма потерь. Обучение и оценка используют предоставленный период, отдельной неизвестной выборки нет.

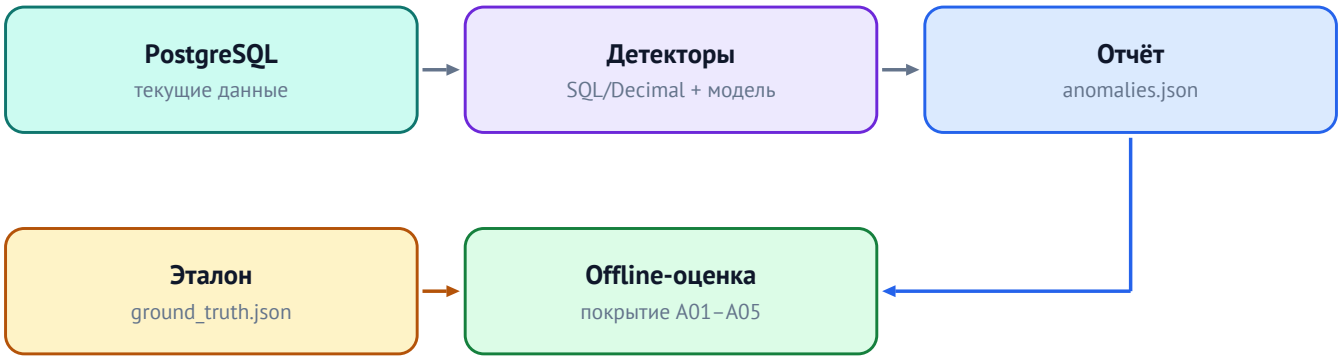
```
uv run python -m app.analytics.anomalies \
  --start 2026-01-01 --end 2026-10-01 \
  --output /tmp/anomalies.json
```

Чтение — REPEATABLE READ READ ONLY, по DATABASE_URL. Без --output JSON печатается в терминал; существующий файл не перезаписывается. В отчёте: период, seed, счётчики наблюдений, настройки и findings с методом и доказательствами.

> **Кандидат — не причина.** Нулевой остаток товара и меньшие продажи рядом по времени ещё не доказывают связь. Дополнительные кандидаты требуют проверки исходных строк.

29 Отдельная оценка сценариев

Эталон читает только отдельная offline-команда оценки.




```
uv run python -m app.analytics.evaluate \
  --report /tmp/anomalies.json \
  --ground-truth data/demo/ground_truth.json
```

Отчёт и эталон должны относиться к одному набору. Совпадения проверяются по периоду, типу сигнала и измерениям; для месячных сценариев также учитывается направление изменения. Полный локальный запуск 21.10.2026: **33 кандидата, 5/5 сценариев**.

Сценарий	Проверяемое событие
A01	Насосы: количество, сентябрь.
A02	Арматура, South: скидки, август.
A03	Компрессоры: возвраты, июль.
A04	Один насос: ноль на складе, 10–19 сентября.
A05	Датчики: позиция на 500 единиц, июнь.

7 кандидатов по месячной статистике + 1 IQR + 1 складской + 24 Isolation Forest. Сценарии покрыты статистикой и правилами; это не оценка отдельно взятой модели. Два запуска с seed 42 дали одинаковый JSON.

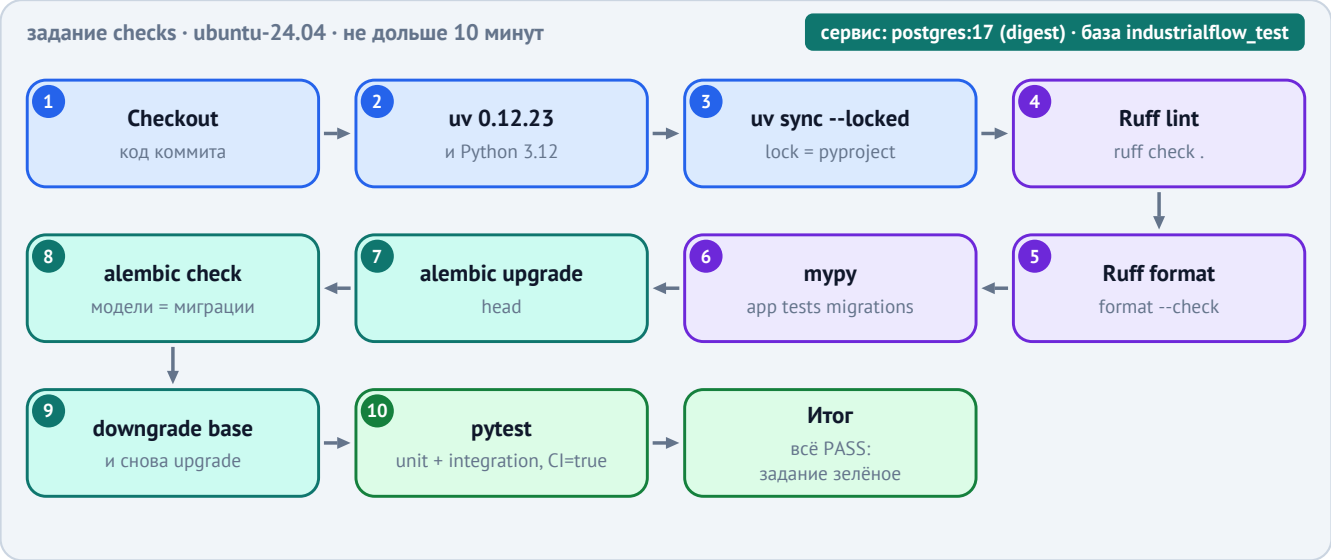
Покрытие ≠ precision. Эталон содержит пять внедрённых положительных сценариев, но не размечает все обычные наблюдения. Даже 5/5 не означает точность 100% или качество на будущих данных. Дополнительный кандидат не становится автоматически ложным.

Проверки: положительные и обычные примеры, MAD = 0, короткая история, IQR, складские пропуски, повторяемость модели и точные SQL-признаки. Интеграционные тесты — только TEST_DATABASE_URL с суффиксом _test. Результаты: раздел 34 и docs/VERIFICATION.md. Дальше — REST API, этап 06.

30 CI на GitHub Actions

Те же проверки на чистой Linux-машине с отдельной PostgreSQL.

Запуск: push в main · pull request в main · вручную: workflow_dispatch



Настройка	Зачем
`services: postgres`	Отдельная база industrialflow_test на время запуска; образ по digest, как в compose.yaml.
`DATABASE_URL, TEST_DATABASE_URL`	Alembic и тесты работают с одной базой *_test.
`CI=true`	Интеграционные тесты без адреса падают, а не пропускаются.
`permissions: contents: read`	Заданию разрешено только читать код.
`checkout@3d3c42e… # v7.0.1`	Действия закреплены полным SHA: тег можно перенести, SHA — нет.
`concurrency`	Новый push в тот же PR отменяет устаревшую проверку.

Статус удалённого CI: не подтверждён. Этап 05 проверен локально поверх 1aa1d24. Результат удалённого GitHub Actions для итогового коммита не подтверждён; приватный репозиторий недоступен через подключённый GitHub API.

31 Шпаргалка команд

Основные команды этапов 01–05.

Задача	Команда
Создать окружение по lock	`uv sync --locked`
Запустить PostgreSQL	`docker compose up --detach --wait db`
Применить миграции	`uv run alembic upgrade head`
Создать demo	`uv run python -m app.data.generate --output data/demo`
Загрузить demo	`uv run python -m app.data.load --input data/demo`
Качество данных	`uv run python -m app.data.quality`
Показатели сентября	`uv run python -m app.analytics --start 2026-09-01 --end 2026-10-01`
Кандидаты на аномалии	`uv run python -m app.analytics.anomalies --start 2026-01-01 --end 2026-10-01`
Запустить сервер	`uv run uvicorn app.main:app --reload`
Проверить готовность	`curl -i http://127.0.0.1:8000/ready`
Тестовая база	`docker compose --profile test up --detach --wait db-test`
Все тесты	`TEST_DATABASE_URL=... uv run pytest`
Только интеграционные	`uv run pytest -m integration`
Линтер и формат	`uv run ruff check . && uv run ruff format --check .`
Типы	`uv run mypy app tests migrations`
Оценка сценариев	`uv run python -m app.analytics.evaluate --help`
Модели = миграции?	`uv run alembic check`
Откатить одну миграцию	`uv run alembic downgrade -1`
Версия схемы	`uv run alembic current`
Остановить базу	`docker compose stop`
Проверить lock	`uv lock --check`
Пересобрать PDF	`uv run --with 'reportlab==4.4.9' python scripts/build_guide.py`

32 Типичные ошибки

Симптом, причина и что сделать.

Симптом	Причина	Что сделать
`uv: command not found`	Терминал открыт до установки uv	Новая вкладка Terminal
`No module named 'app'`	Нет pythonpath или команда не из корня	Проверить pyproject.toml, cd в корень
`Address already in use`	Порт занят	`lsof -i :8000`, `--port 8001`
`405 Method Not Allowed`	Не тот HTTP-метод	Для /health — GET
`uv sync --locked` падает	pyproject изменён, lock — нет	`uv lock` и коммит uv.lock
Изменения .env не видны	Настройки читаются при старте	Перезапустить сервер
`failed to connect to the docker API`	Docker Desktop не запущен	Запустить Docker Desktop
`POSTGRES_PASSWORD is missing`	Нет .env	`cp .env.example .env`
`port is already allocated`	Порт 5436 занят	Сменить POSTGRES_PORT и DATABASE_URL
`DATABASE_URL не задан`	Нет .env или переменной	Создать .env
/ready → 503 при живой базе	Миграции не применены	`uv run alembic upgrade head`
`New upgrade operations detected`	Модель изменена без миграции	Новая миграция --autogenerate
Интеграционные тесты skipped	Нет TEST_DATABASE_URL	Запустить db-test, задать переменную
«только для базы с суффиксом _test»	Адрес указывает на другую базу	Указать базу *_test
`IntegrityError`	Нарушено правило целостности	Имя ограничения в тексте ошибки
Данные пропали	`docker compose down -v`	Без -v том сохраняется

33 Проверь себя

Вопросы для самопроверки и самостоятельные задания.

1. Чем pyproject.toml отличается от uv.lock?
Ответ: pyproject — требования к прямым зависимостям, uv.lock — точные версии и хэши всех пакетов.

2. Почему Uvicorn находит app, а pytest без настройки — нет?
Ответ: Uvicorn добавляет текущую папку в sys.path, pytest — папку tests/. pythonpath = ["."] добавляет корень.

3. Что будет, если health() вернёт {"status": "fail"}?
Ответ: mypy найдёт ошибку до запуска; при запросе FastAPI выбросит ResponseValidationError, клиент получит 500.

4. Почему заказ хранится в двух таблицах?
Ответ: Один заказ — много товаров; нормализация хранит свойства заказа один раз, а товары — построчно.

5. Чем CASCADE отличается от RESTRICT?
Ответ: CASCADE удаляет зависимые строки вместе с родителем; RESTRICT запрещает удалить родителя со ссылками.

6. Почему деньги — NUMERIC и Decimal?
Ответ: float двоичный и не хранит 0.1 точно; NUMERIC и Decimal хранят десятичные числа точно.

7. Зачем AT TIME ZONE 'Europe/Moscow'?
Ответ: Отчётный месяц московский: оплата 1 октября в 01:30 МСК по UTC попала бы в сентябрь.

8. Почему применённую миграцию не редактируют?
Ответ: Иначе разные базы окажутся с одинаковым номером версии; изменение — новая миграция.

9. Почему JOIN с возвратами завышает продажи?
Ответ: Позиция повторяется на каждый возврат; нужно сначала свести таблицу «многих» к строке на ключ.

10. Чем /ready отличается от /health?
Ответ: /health — процесс жив, база не нужна; /ready — база доступна и схема актуальна, иначе 503.

Самостоятельные задания

Этап 01. Маршрут GET /version с моделью VersionResponse: name — из get_settings().app_name, version — из app.__version__; тест на 200 и тело ответа.

Этап 02. Столбец products.is_active (boolean, NOT NULL, по умолчанию true) миграцией 0002 с server_default; alembic check молчит, откат downgrade -1 работает, есть тест.

34 Итоги проверки

Дата проверки: 2026-10-21; база рабочего дерева 1aa1d24 + worktree.

Окружение: macOS, Apple Silicon, uv 0.12.23, Python 3.12.15, Docker 29.8.2, PostgreSQL 17. Тесты: 223 прошли (136 unit + 87 integration), 0 упали, 0 пропущены.

Проверка	Команда	Результат	Статус
Тесты	`CI=true TEST_DATABASE_URL=... pytest`	223 passed: 136 unit + 87 integration	**PASS**
Линтер	`ruff check .`	All checks passed	**PASS**
Формат	`ruff format --check .`	Python и блоки Markdown	**PASS**
Типы	`mypy app tests migrations`	Нет ошибок в 46 файлах	**PASS**
Миграции	`alembic upgrade / check / downgrade`	Схема 0001; расхождений нет	**PASS**
Зависимости	`uv lock --check --offline`	52 пакета; newest upload 2026-10-03	**PASS**
Полный детектор	`python -m app.analytics.anomalies`	33 кандидата: 7 robust, 1 IQR, 1 stock, 24 IF	**PASS**
Сценарии	`python -m app.analytics.evaluate`	Покрытие A01–A05: 5/5; точность не измерена	**PASS**
Повторяемость	`same data + seed 42, two runs`	JSON-отчёты совпали; 0 нарушений качества	**PASS**

Ответы сервера

Запрос	Код	Тело
`GET /health`	200 OK	`{"status":"ok"}`
`GET /ready`	200 OK	`{"status":"ready","revision":"0001"}`
`POST /health`	405 Method Not Allowed	`{"detail":"Method Not Allowed"}, allow: GET`
`GET /nope`	404 Not Found	`{"detail":"Not Found"}`
`GET /docs`	200 OK	`HTML-страница Swagger UI`

GitHub Actions: не подтверждён. Этап 05 проверен локально поверх 1aa1d24. Результат удалённого GitHub Actions для итогового коммита не подтверждён; приватный репозиторий недоступен через подключённый GitHub API.

Что дальше

- Этап 06: REST API для точных показателей, сравнений и поиска аномалий.
- Подтвердить GitHub Actions для итогового коммита; локальный прогон не заменяет удалённый CI.
- Оценены пять положительных сценариев; ложные срабатывания, причинность и качество на новых данных не установлены.